

RE Leçon 1 — Exercices pratiques

Setup

```
mkdir /tmp/c-lessons && cd /tmp/c-lessons
```

Exercice 1 — Variables et affichage

Objectif : déclarer des variables et les afficher.

Écris un programme qui :

- Déclare un entier `score` = 42
- Déclare un caractère `niveau` = 'A'
- Déclare une chaîne `joueur` = "Alice"
- Affiche les trois valeurs

```
nano ex1.c  
gcc -o ex1 ex1.c  
./ex1
```

Résultat attendu :

```
joueur: Alice  
score: 42  
niveau: A
```

► Solution

Exercice 2 — Fonction simple

Objectif : écrire et appeler une fonction.

Écris un programme avec une fonction `multiplier(int a, int b)` qui retourne le produit de deux entiers. Affiche le résultat de `multiplier(6, 7)`.

Résultat attendu :

```
6 x 7 = 42
```

► Solution

Exercice 3 — Condition (crackme niveau 0)

Objectif : programme qui vérifie un code secret.

Écris un programme qui :

- Demande à l'utilisateur d'entrer un nombre
- Si le nombre est 1337 → affiche "Access granted"
- Sinon → affiche "Access denied"

```
gcc -o ex3 ex3.c
./ex3
```

Résultat attendu :

```
Code: 1337
Access granted
```

► Solution

Exercice 4 — Boucle

Objectif : afficher une table de multiplication.

Écris un programme qui affiche la table de multiplication de 3 (de 1 à 10).

Résultat attendu :

```
3 x 1 = 3
3 x 2 = 6
...
3 x 10 = 30
```

► Solution

Exercice 5 — Pointeurs

Objectif : comprendre les pointeurs.

Écris un programme qui :

- Déclare un entier `x = 10`
- Crée un pointeur `p` qui pointe vers `x`

- Affiche l'adresse de `x` via `p`
- Modifie `x` à `99` via le pointeur
- Affiche la nouvelle valeur de `x`

Résultat attendu :

```
adresse de x : 0x7fff...  
valeur avant : 10  
valeur après : 99
```

► Solution

Exercice 6 — Strings

Objectif : manipuler des chaînes de caractères.

Écris un programme qui :

- Déclare `prenom[] = "Alice"` et `nom[] = "Dupont"`
- Affiche la longueur de chaque chaîne
- Concatène les deux dans `complet` avec un espace entre les deux
- Affiche `complet`

Résultat attendu :

```
longueur prenom : 5  
longueur nom : 6  
complet : Alice Dupont
```

► Solution

Exercice 7 — Struct

Objectif : créer et utiliser une struct.

Crée une struct `Joueur` avec :

- `nom` (chaîne de 32 caractères)
- `score` (entier)
- `vip` (entier : 0 ou 1)

Dans `main` :

- Crée un joueur avec `nom="Bob"`, `score=500`, `vip=0`
- Affiche ses infos
- Passe `vip` à 1 et réaffiche

Résultat attendu :

```
Joueur: Bob | Score: 500 | VIP: non  
Joueur: Bob | Score: 500 | VIP: oui
```

► Solution

Exercice 8 — Crackme (RE orienté)

Objectif : trouver le mot de passe d'un binaire sans regarder le code.

Compile ce programme **sans regarder le code source après** :

```
// crackme1.c  
#include <stdio.h>  
#include <string.h>  
  
int main() {  
    char input[50];  
    printf("Password: ");  
    scanf("%49s", input);  
    if (strcmp(input, "m4tr1x") == 0) {  
        printf("Flag: RE_{bien_joue}\n");  
    } else {  
        printf("Wrong password\n");  
    }  
    return 0;  
}
```

```
gcc -o crackme1 crackme1.c
```

Maintenant trouve le mot de passe sans le regarder dans le code :

```
strings crackme1
```

Cherche une chaîne qui ressemble à un mot de passe dans la sortie, puis teste-la :

```
./crackme1
```

C'est exactement ce qu'on fait en reverse engineering sur un vrai binaire.

Checkpoint — tu dois avoir fait

- ☐ Ex 1 : variables compilé et exécuté
- ☐ Ex 2 : fonction compilée et exécutée
- ☐ Ex 3 : condition — programme qui demande un code
- ☐ Ex 4 : boucle — table de multiplication
- ☐ Ex 5 : pointeurs — modifier via adresse
- ☐ Ex 6 : strings — concat et longueur
- ☐ Ex 7 : struct — joueur VIP
- ☐ Ex 8 : crackme — mot de passe trouvé avec `strings`